



```

import A5O
import Closure
import Hops
set_option autoImplicit false

```

```

/-! # A5O — Ten closures over the objects on record (2026-09-19).
Imports v4 (0e5ebe3f...), Closure (af6b3233...), Hops (ca7a950d...) unmodified.
Numbering follows the Chairman's list of 2026-09-19. Each section states
the ledger rule it consumes (if any) as a named hypothesis. -/

```

```

namespace Ten

```

```

variable {Party Action AoC : Type}
variable (C : Party → Party → Action → Nat → Prop)
variable (I : Party → Prop)
variable (M : Party → Action → Prop)
variable (S : Party → Action → Prop)
variable (R : Party → Party → Action → Nat → Prop)
variable (Appointed : Party → Party → Action → Nat → Prop)
variable (Revoke : Party → Party → Action → Nat → Prop)
variable (Executes : Party → Action → Nat → Prop)
variable (inAoC : Action → AoC → Prop)
variable (appointAct : Party → Action → Action)
variable (Root : Party → Prop)

```

```

open Hops

```

```

-- =====
-- Shared: a trace that remembers its root (Hops.Traces with the root as
-- an index). Same two constructors, same O-instance at every link.
-- =====

```

```

inductive Reaches : Party → Action → Nat → Party → Prop
| root (P A : Party) (X : Action) (t : Nat) :
  Root P → O C I M S R Appointed P A X t → Reaches A X t P
| hop (P A : Party) (X : Action) (s t : Nat) (Rt : Party) :
  s < t → O C I M S R Appointed P A X t →
  Reaches P (appointAct A X) s Rt → Reaches A X t Rt

```

```

theorem traces_reaches (A : Party) (X : Action) (t : Nat)
  (h : Traces C I M S R Appointed appointAct Root A X t) :
  ∃ Rt, Reaches C I M S R Appointed appointAct Root A X t Rt := by
  induction h with
  | root P A X t hR hO => exact ⟨P, Reaches.root P A X t hR hO⟩
  | hop P A X s t hs hO _ ih =>

```

```

obtain ⟨Rt, hr⟩ := ih
exact ⟨Rt, Reaches.hop P A X s t Rt hs hO hr⟩

```

```

-- =====
-- 1. SCOPE CANNOT EXPAND ACROSS A HOP
-- =====

```

```

section Scope

```

```

variable (cap : Party → Action → Nat) -- a party's ceiling over an action

```

```

/-- H6. A grant confers no more than the appointer holds over the same act. -/

```

```

def Attenuating : Prop :=

```

```

  ∀ (P A : Party) (X : Action) (t : Nat), Appointed P A X t → cap A X ≤ cap P X

```

```

/-- H7. The ceiling to appoint over X is the ceiling over X: authority to
confer is bounded by authority held. -/

```

```

def ConferBounded : Prop :=

```

```

  ∀ (P A : Party) (X : Action), cap P (appointAct A X) = cap P X

```

```

/-- At every hop depth, the acting party's ceiling is within the root's. -/

```

```

theorem scope_attenuates_to_root

```

```

  (h6 : Attenuating Appointed cap) (h7 : ConferBounded appointAct cap)

```

```

  (A : Party) (X : Action) (t : Nat) (Rt : Party)

```

```

  (h : Reaches C I M S R Appointed appointAct Root A X t Rt) :

```

```

  cap A X ≤ cap Rt X := by

```

```

  induction h with

```

```

  | root P A X t _ hO => exact h6 P A X t hO.2.2.2.2.2

```

```

  | hop P A X s t Rt _ hO _ ih =>

```

```

    have h1 : cap A X ≤ cap P X := h6 P A X t hO.2.2.2.2.2

```

```

    have h2 : cap P X ≤ cap Rt X := by

```

```

      have := ih

```

```

      rw [h7 P A X, h7 Rt A X] at this

```

```

      exact this

```

```

    exact Nat.le_trans h1 h2

```

```

/-- Kestrel `logistics-tier2`: a €75,000 sub-agent under a €50,000 parent
cannot be an appointment in an attenuating ledger. -/

```

```

theorem kestrel_impossible

```

```

  (h6 : Attenuating Appointed cap)

```

```

  (P A : Party) (X : Action) (t : Nat)

```

```

  (hParent : cap P X = 50000) (hChild : cap A X = 75000) :

```

```

  ¬ Appointed P A X t := by

```

```

  intro hApp

```

```

  have := h6 P A X t hApp

```

```

  rw [hParent, hChild] at this

```

exact absurd this (by decide)

end Scope

```
-- =====
-- 2. THE WHOLE TRACE IS UNIQUE — one execution, one root
-- =====

/-- H8. A root is never a grantee. -/
def RootNotAppointed : Prop :=
  ∀ (P : Party), Root P → ∀ (Q : Party) (Y : Action) (s : Nat), ¬ Appointed Q P Y s

/-- Under one-appointer-per-grant and roots-are-not-grantees, any two
traces of the same agent for the same act — at any two times — reach the
same root. The trace is a function of the execution. -/
theorem root_unique
  (h4 : OneAppointer Appointed) (h8 : RootNotAppointed Appointed Root)
  (A : Party) (X : Action) (t : Nat) (Rt : Party)
  (h : Reaches C I M S R Appointed appointAct Root A X t Rt) :
  ∀ (t' : Nat) (Rt' : Party),
    Reaches C I M S R Appointed appointAct Root A X t' Rt' → Rt = Rt' := by
  induction h with
  | root P A X t hRootP hO =>
    intro t' Rt' h'
    rcases h' with ⟨_, _, _, _, hO'⟩ | ⟨P', _, _, s', _, _, hO', hr'⟩
    · exact h4 P Rt' A X t t' hO.2.2.2.2.2 hO'.2.2.2.2.2
    · have hPP : P = P' := h4 P P' A X t t' hO.2.2.2.2.2 hO'.2.2.2.2.2
      rw [← hPP] at hr'
      rcases hr' with ⟨_, _, _, _, hOQ⟩ | ⟨Q, _, _, _, _, hOQ, _⟩
      · exact absurd hOQ.2.2.2.2.2 (h8 P hRootP _)
      · exact absurd hOQ.2.2.2.2.2 (h8 P hRootP Q _)
  | hop P A X s t Rt hs hO hr ih =>
    intro t' Rt' h'
    rcases h' with ⟨_, _, _, _, hRootP', hO'⟩ | ⟨P', _, _, s', _, _, hO', hr'⟩
    · have hPP : P = Rt' := h4 P Rt' A X t t' hO.2.2.2.2.2 hO'.2.2.2.2.2
      rw [← hPP] at hRootP'
      rcases hr with ⟨_, _, _, _, hOQ⟩ | ⟨Q, _, _, _, _, hOQ, _⟩
      · exact absurd hOQ.2.2.2.2.2 (h8 P hRootP' _)
      · exact absurd hOQ.2.2.2.2.2 (h8 P hRootP' Q _)
    · have hPP : P = P' := h4 P P' A X t t' hO.2.2.2.2.2 hO'.2.2.2.2.2
      rw [← hPP] at hr'
      exact ih s' Rt' hr'

-- =====
-- 3. ACYCLICITY IN TIME — no link is its own ancestor
```

```
-- =====
```

```
/-- Every ancestor link in a trace is strictly earlier than the link below
it; in particular the trace is finite and a link cannot recur in its own
ancestry. Stated as: the hop's antecedent time is strictly less. -/
```

```
theorem ancestors_strictly_earlier
```

```
(P A : Party) (X : Action) (s t : Nat) (Rt : Party)
(hs : s < t) (hO : O C I M S R Appointed P A X t)
(hr : Reaches C I M S R Appointed appointAct Root P (appointAct A X) s Rt) :
Reaches C I M S R Appointed appointAct Root A X t Rt ∧ s < t :=
⟨Reaches.hop P A X s t Rt hs hO hr, hs⟩
```

```
/-- Ancestry along a trace: (P, appointAct A X, s) is an ancestor link of
(A, X, t) when a hop joins them, transitively. -/
```

```
inductive AncestorOf : Party → Action → Nat → Party → Action → Nat → Prop
```

```
| step (P A : Party) (X : Action) (s t : Nat) :
  s < t → O C I M S R Appointed P A X t → AncestorOf P (appointAct A X) s A X t
| trans (B Y u A X t P Z s : _) :
  AncestorOf P Z s B Y u → AncestorOf B Y u A X t → AncestorOf P Z s A X t
```

```
theorem ancestor_earlier (P : Party) (Z : Action) (s : Nat) (A : Party) (X : Action) (t : Nat)
```

```
(h : AncestorOf C I M S R Appointed appointAct P Z s A X t) : s < t := by
```

```
induction h with
```

```
| step _ _ _ _ _ hs _ => exact hs
| trans _ _ _ _ _ _ _ _ _ ih1 ih2 => exact Nat.lt_trans ih1 ih2
```

```
/-- NO CYCLES: no link is its own ancestor. -/
```

```
theorem no_link_is_own_ancestor (A : Party) (X : Action) (t : Nat) :
```

```
¬ AncestorOf C I M S R Appointed appointAct A X t A X t :=
```

```
fun h => Nat.lt_irrefl t (ancestor_earlier C I M S R Appointed appointAct A X t A X t h)
```

```
-- =====
```

```
-- 4. EVERY PRINCIPAL IN A TRACE IS IDENTIFIED
```

```
-- =====
```

```
theorem root_identified
```

```
(A : Party) (X : Action) (t : Nat) (Rt : Party)
(h : Reaches C I M S R Appointed appointAct Root A X t Rt) : I Rt := by
induction h with
| root _ _ _ _ _ hO => exact hO.2.1
| hop _ _ _ _ _ _ _ _ _ ih => exact ih
```

```
theorem immediate_identified
```

```
(A : Party) (X : Action) (t : Nat) (Rt : Party)
(h : Reaches C I M S R Appointed appointAct Root A X t Rt) :
∃ P, O C I M S R Appointed P A X t ∧ I P := by
```

```

rcases h with ⟨_, _, _, _, _, hO⟩ | ⟨P, _, _, _, _, _, hO, _⟩
· exact ⟨Rt, hO, hO.2.1⟩
· exact ⟨P, hO, hO.2.1⟩

```

-- Every link's principal is identified: stated as an inductive "all links" predicate and proved for every trace. -/

```

inductive AllIdentified : Party → Action → Nat → Prop
| root (P A : Party) (X : Action) (t : Nat) :
  | P → AllIdentified A X t
| hop (P A : Party) (X : Action) (s t : Nat) :
  | P → AllIdentified P (appointAct A X) s → AllIdentified A X t

```

theorem trace_all_identified

```

(A : Party) (X : Action) (t : Nat) (Rt : Party)
(h : Reaches C I M S R Appointed appointAct Root A X t Rt) :
AllIdentified I appointAct A X t := by
induction h with
| root P A X t _ hO => exact AllIdentified.root P A X t hO.2.1
| hop P A X s t _ _ hO _ ih => exact AllIdentified.hop P A X s t hO.2.1 ih

```

```

-- =====
-- 5. A RECEIPT IS NOT A PASS — O at t does not carry to t' > t
-- =====

```

section Receipt

```

def rC : Unit → Unit → Unit → Nat → Prop := fun _ _ _ => True
def rI : Unit → Prop := fun _ => True
def rM : Unit → Unit → Prop := fun _ _ => True
def rS : Unit → Unit → Prop := fun _ _ => True
-- Revocation dominance stands only at time 0; revoked from 1 onward. -/
def rR : Unit → Unit → Unit → Nat → Prop := fun _ _ _ t => t = 0
def rApp : Unit → Unit → Unit → Nat → Prop := fun _ _ _ => True

```

theorem receipt_at_0 : O rC rI rM rS rR rApp () () () 0 :=
⟨trivial, trivial, trivial, trivial, rfl, trivial⟩

theorem no_O_at_1 : ¬ O rC rI rM rS rR rApp () () () 1 :=
fun h => Nat.noConfusion h.2.2.2.1

-- The general persistence principle is FALSE: a receipt at t does not entail O at a later t'. -/

theorem receipt_not_capability :

```

¬ (∀ (C : Unit → Unit → Unit → Nat → Prop) (I : Unit → Prop)
  (M S : Unit → Unit → Prop)
  (R Ap : Unit → Unit → Unit → Nat → Prop) (t t' : Nat),

```

```

      O C I M S R Ap () () t → t < t' → O C I M S R Ap () () t') :=
    fun h => no_O_at_1 (h rC rl rM rS rR rApp 0 1 receipt_at_0 (Nat.lt_succ_self 0))
end Receipt

```

```

-- =====
-- 6. DENY IS ALSO DECIDED — each refuse path entails ¬O now
-- =====

```

```

theorem deny_no_grant (P A : Party) (X : Action) (t : Nat)
  (h : ¬ Appointed P A X t) : ¬ O C I M S R Appointed P A X t :=
  N6_necessity C I M S R Appointed P A X t h

```

```

theorem deny_out_of_scope (P A : Party) (X : Action) (t : Nat)
  (h : ¬ S A X) : ¬ O C I M S R Appointed P A X t :=
  N4_necessity C I M S R Appointed P A X t h

```

```

theorem deny_revoked (P A : Party) (X : Action) (t : Nat)
  (h : ¬ R P A X t) : ¬ O C I M S R Appointed P A X t :=
  N5_necessity C I M S R Appointed P A X t h

```

/-- A gated ADA system never enforces any of the three. -/

```

theorem deny_blocks_enforcement
  (G : GatedSystem Executes inAoC)
  (hADA : achievesADA C I M S R Appointed inAoC G.sys)
  (P A : Party) (X : Action) (t : Nat) (d : AoC) (hd : inAoC X d)
  (h : ¬ Appointed P A X t ∨ ¬ S A X ∨ ¬ R P A X t) :
  ¬ G.sys.enforces P A X t :=
  fun hEnf =>
    let hO := hADA P A X t d hd hEnf
    match h with
    | Or.inl h1 => h1 hO.2.2.2.2
    | Or.inr (Or.inl h2) => h2 hO.2.2.2.1
    | Or.inr (Or.inr h3) => h3 hO.2.2.2.2.1

```

```

-- =====
-- 7. REVOKED MEANS NO REVIVAL OF THAT TRIPLE
-- =====

```

/-- Once Q has revoked P for Y (after having appointed), the triple (Q, P, Y) never satisfies O again at any later time — regardless of any later grant carrying the same triple. A revival is therefore a distinct appointment (a new `appointment\_ref`), which this model represents as a distinct act identity, not a re-use of Y. -/

```

theorem revocation_permanent
  (hP2 : P2 R Appointed Revoke) (h5 : RevokePersists Revoke)
  (Q P : Party) (Y : Action) (s t' : Nat)

```

```

(hApp : Appointed Q P Y s) (hs : s < t') (hRev : Revoke Q P Y t') :
  ∀ t'', t' ≤ t'' → ¬ O C I M S R Appointed Q P Y t'' :=
fun t'' hle hO =>
  hP2 Q P Y s t'' (Nat.lt_of_lt_of_le hs hle) hApp
  (h5 Q P Y t' t'' hle hRev) hO.2.2.2.1

-- =====
-- 8. THE CORPORATE PRINCIPAL RESTORES UNIQUENESS
-- =====

section Corporate
variable {Prin : Type}
variable (Cq : Prin → Party → Action → Nat → Prop) -- co-signer level
variable (legalPerson : Prin → Party) -- resolver to the entity

/-- The board's signers for one execution all resolve to one legal person. -/
def ResolvesToOne : Prop :=
  ∀ (q q' : Prin) (A : Party) (X : Action) (t : Nat),
    Cq q A X t → Cq q' A X t → legalPerson q = legalPerson q'

/-- The corporate binding: the entity is the signer of record. -/
def corpC : Party → Party → Action → Nat → Prop :=
  fun P A X t => ∃ q, Cq q A X t ∧ legalPerson q = P

theorem corporate_C_functional (h : ResolvesToOne Cq legalPerson) :
  CFunctional (corpC Cq legalPerson) := by
  intro P P' A X t hP hP'
  obtain ⟨q, hq, rfl⟩ := hP
  obtain ⟨q', hq', rfl⟩ := hP'
  exact h q q' A X t hq hq'

/-- O_unique returns for the entity, over v4's own O. -/
theorem corporate_O_unique (h : ResolvesToOne Cq legalPerson)
  (I : Party → Prop) (M S : Party → Action → Prop)
  (R Ap : Party → Party → Action → Nat → Prop) :
  ∀ (P P' A : Party) (X : Action) (t : Nat),
    O (corpC Cq legalPerson) I M S R Ap P A X t →
    O (corpC Cq legalPerson) I M S R Ap P' A X t → P = P' :=
  O_unique (corpC Cq legalPerson) I M S R Ap (corporate_C_functional Cq legalPerson h)

/-- v4's own quorum instance (same content as `quorumC`, over Nat time),
resolved: a and b both act for x, the entity. -/
def qC : QuorumPrin → QuorumPrin → Unit → Nat → Prop := fun p _ _ => p = QuorumPrin.a
  ∨ p = QuorumPrin.b
def boardToX : QuorumPrin → QuorumPrin := fun _ => QuorumPrin.x

```

```

theorem v4_quorum_two_governors_nat : qC QuorumPrin.a QuorumPrin.x () 0  $\wedge$  qC
QuorumPrin.b QuorumPrin.x () 0 :=
  (Or.inl rfl, Or.inr rfl)

```

```

theorem v4_quorum_resolves : ResolvesToOne (Party := QuorumPrin) (Action := Unit) qC
boardToX :=
  fun _ _ _ _ _ => rfl

```

```

theorem v4_quorum_corporate_functional :
  CFunctional (corpC (Party := QuorumPrin) (Action := Unit) qC boardToX) :=
  corporate_C_functional (Party := QuorumPrin) (Action := Unit) _ v4_quorum_resolves
end Corporate

```

```

-- =====
-- 9. LINEARIZABILITY OF DECISIONS over the ledger's total order
-- =====

```

```

section Linear

```

```

variable {Grant : Type}

```

```

-- revokedSeq g = some r : grant g was revoked at ledger seq r (append-only).

```

```

variable (revokedSeq : Grant  $\rightarrow$  Option Nat)

```

```

/-- The gate allows g at ledger head h iff no revocation of g has seq  $\leq$  h.

```

```

`revocation_as_of` on the receipt is h. -/

```

```

def Allow (g : Grant) (h : Nat) : Prop :=

```

```

   $\forall$  r, revokedSeq g = some r  $\rightarrow$  h < r

```

```

/-- Every revocation the head could see was seen. -/

```

```

theorem allow_sees_all_prior (g : Grant) (h r : Nat)

```

```

  (hA : Allow revokedSeq g h) (hr : r  $\leq$  h) : revokedSeq g  $\neq$  some r :=

```

```

  fun heq => Nat.lt_irrefl r (Nat.lt_of_le_of_lt hr (hA r heq))

```

```

/-- Decisions are monotone in the head: an ALLOW at h was an ALLOW at any
earlier head. -/

```

```

theorem allow_monotone (g : Grant) (h h' : Nat)

```

```

  (hA : Allow revokedSeq g h) (hle : h'  $\leq$  h) : Allow revokedSeq g h' :=

```

```

  fun r heq => Nat.lt_of_le_of_lt hle (hA r heq)

```

```

/-- Once revoked at r, no head  $\geq$  r allows. Revocation wins in the order. -/

```

```

theorem revoked_never_allowed (g : Grant) (r h : Nat)

```

```

  (hrev : revokedSeq g = some r) (hle : r  $\leq$  h) :  $\neg$  Allow revokedSeq g h :=

```

```

  fun hA => allow_sees_all_prior revokedSeq g h r hA hle hrev

```

```

/-- Two receipts for the same grant are consistent with one total order:

```

```

if an ALLOW at  $h_1$  and a DENY-by-revocation at  $h_2$  both exist,  $h_1 < h_2$ . -/
theorem allow_before_revocation (g : Grant) (h1 h2 r : Nat)
  (hA : Allow revokedSeq g h1) (hrev : revokedSeq g = some r) (hle : r ≤ h2) :
  h1 < h2 :=
  Nat.lt_of_lt_of_le (hA r hrev) hle
end Linear

```

```

-- =====
-- 10. NO SEVENTH CONJUNCT — the necessary conditions on a governor are
--   exactly those O entails
-- =====
section Fixed
variable (AoC)

```

```

/-- Q is NECESSARY if, across every gated ADA system and every area
predicate, every governor the system enforces satisfies Q. -/

```

```

def Necessary (Q : Party → Party → Action → Nat → Prop) : Prop :=
  ∀ (inAoC' : Action → AoC → Prop)
    (sys : ADASystem Party Party Action Nat),
    achievesADA C I M S R Appointed inAoC' sys →
    ∀ (P A : Party) (X : Action) (t : Nat) (d : AoC),
    inAoC' X d → sys.enforces P A X t → Q P A X t

```

```

/-- Q is ENTAILED if O alone proves it. -/

```

```

def Entailed (Q : Party → Party → Action → Nat → Prop) : Prop :=
  ∀ (P A : Party) (X : Action) (t : Nat), O C I M S R Appointed P A X t → Q P A X t

```

```

theorem entailed_necessary (Q : Party → Party → Action → Nat → Prop)
  (h : Entailed C I M S R Appointed Q) : Necessary AoC C I M S R Appointed Q :=
  fun __ hADA P A X t d hd hEnf => h P A X t (hADA P A X t d hd hEnf)

```

```

/-- The canonical monitor enforces exactly O and achieves ADA; a necessary
Q must therefore hold wherever O holds. Needs one inhabitant of AoC. -/

```

```

theorem necessary_entailed (d0 : AoC) (Q : Party → Party → Action → Nat → Prop)
  (h : Necessary AoC C I M S R Appointed Q) : Entailed C I M S R Appointed Q :=
  fun P A X t hO =>
    h (fun __ => True) (fun P' A' X' t' => O C I M S R Appointed P' A' X' t')
    (fun _ _ _ _ _ hE => hE) P A X t d0 trivial hO

```

```

/-- FIXED POINT. A condition is necessary of every governor in every
accountable system if and only if O already entails it. Nothing can be
added to O that is both new and necessary; O is the complete set. -/

```

```

theorem no_seventh_conjunct (d0 : AoC) (Q : Party → Party → Action → Nat → Prop) :
  Necessary AoC C I M S R Appointed Q ↔ Entailed C I M S R Appointed Q :=

```

```

    <necessary_entailed AoC C I M S R Appointed d0 Q, entailed_necessary AoC C I M S R
    Appointed Q>

```

```

/-- Corollary: conjoining any necessary Q to O changes nothing. -/

```

```

theorem seventh_absorbed (d0 : AoC) (Q : Party → Party → Action → Nat → Prop)

```

```

  (h : Necessary AoC C I M S R Appointed Q) (P A : Party) (X : Action) (t : Nat) :

```

```

  (O C I M S R Appointed P A X t ∧ Q P A X t) ↔ O C I M S R Appointed P A X t :=

```

```

  <fun hh => hh.1, fun hO => <hO, necessary_entailed AoC C I M S R Appointed d0 Q h P A X t
  hO>>

```

```

end Fixed

```

```

end Ten

```

```

#print axioms Ten.traces_reaches

```

```

#print axioms Ten.scope_attenuates_to_root

```

```

#print axioms Ten.kestrel_impossible

```

```

#print axioms Ten.root_unique

```

```

#print axioms Ten.ancestors_strictly_earlier

```

```

#print axioms Ten.ancestor_earlier

```

```

#print axioms Ten.no_link_is_own_ancestor

```

```

#print axioms Ten.root_identified

```

```

#print axioms Ten.immediate_identified

```

```

#print axioms Ten.trace_all_identified

```

```

#print axioms Ten.receipt_not_capability

```

```

#print axioms Ten.deny_no_grant

```

```

#print axioms Ten.deny_out_of_scope

```

```

#print axioms Ten.deny_revoked

```

```

#print axioms Ten.deny_blocks_enforcement

```

```

#print axioms Ten.revocation_permanent

```

```

#print axioms Ten.corporate_C_functional

```

```

#print axioms Ten.corporate_O_unique

```

```

#print axioms Ten.v4_quorum_corporate_functional

```

```

#print axioms Ten.allow_sees_all_prior

```

```

#print axioms Ten.allow_monotone

```

```

#print axioms Ten.revoked_never_allowed

```

```

#print axioms Ten.allow_before_revocation

```

```

#print axioms Ten.no_seventh_conjunct

```

```

#print axioms Ten.seventh_absorbed

```